

Multi-language Program Logics

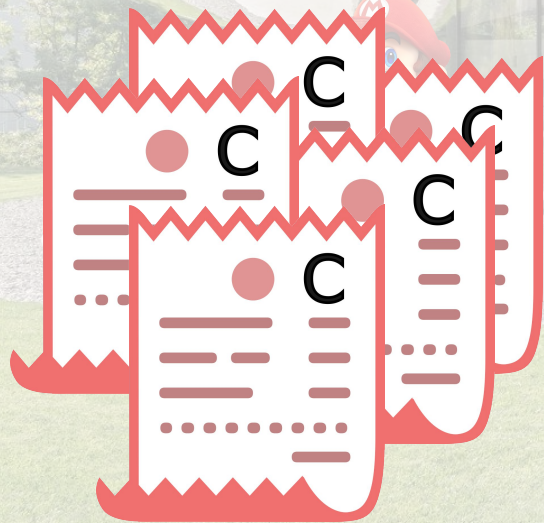
Alex Loitzl

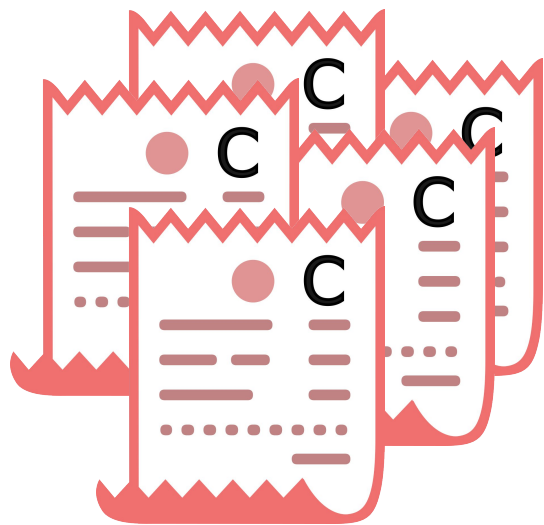


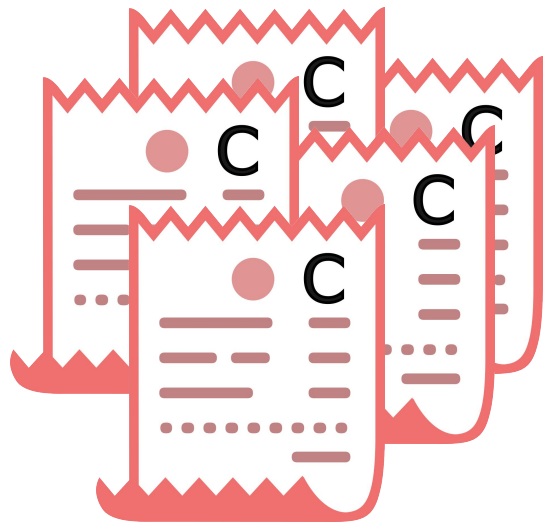
Institute of
Science and
Technology
Austria



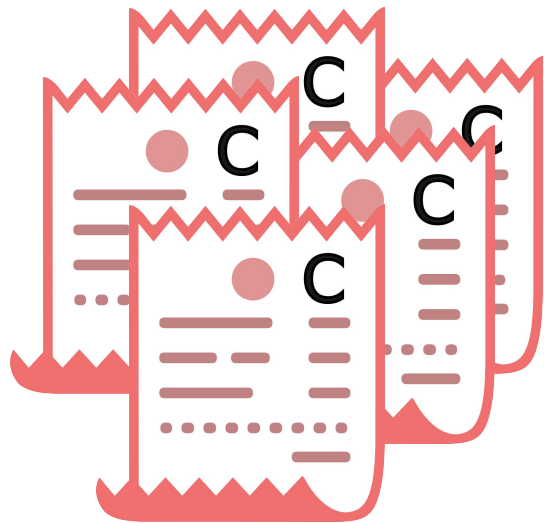
Verify, Wah!







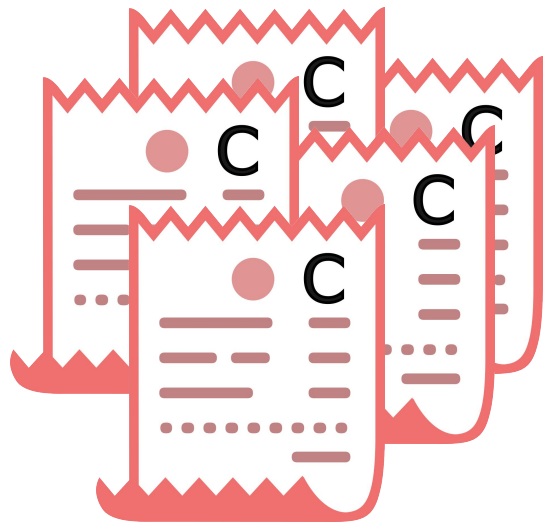
```
echorec( $\ell$ )  $\triangleq$   
  let  $x := \text{getc}()$  in  
  log( $\ell$ ,  $x$ );  
  putc( $x$ )
```



{ ... }

```
echorec(ℓ)  $\triangleq$   
  let x := getc() in  
  log(ℓ, x);  
  putc(x)
```

{ ... }

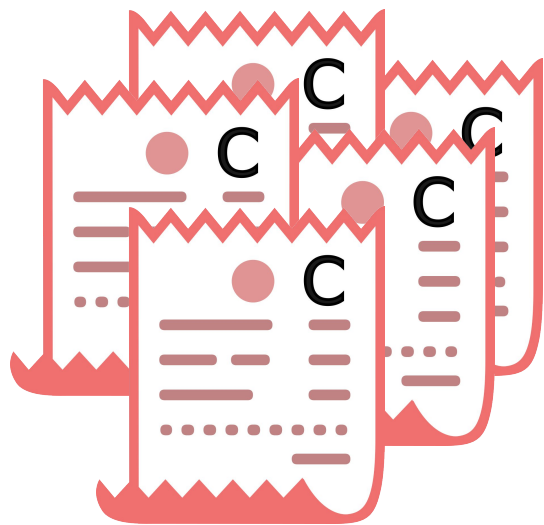


$\{\ell \mapsto buf\}$

$\text{echo}_{\text{rec}}(\ell) \triangleq$

$\text{log}(\ell, x);$

$\{ \dots \}$



$$\{\ell \mapsto buf\}$$

$$echo_{rec}(\ell) \triangleq$$

$$\log(\ell, x);$$

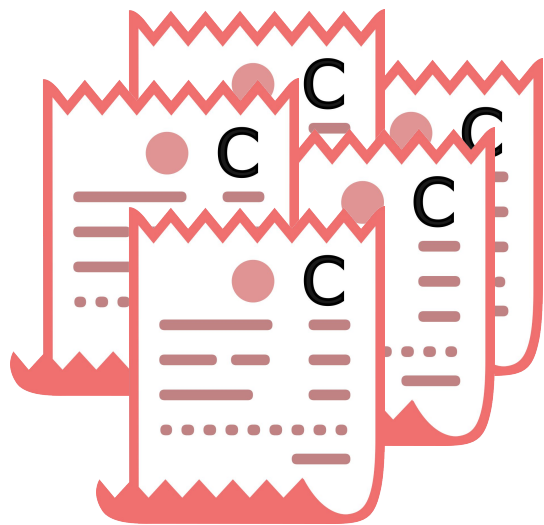
$$\{ \dots \}$$

$$\{\ell \mapsto buf\}$$

$$\log_{rec}(\ell, v) \triangleq$$

$$\vdots$$

$$\{\ell \mapsto v :: buf\}$$



$$\{\ell \mapsto buf\}$$

$$echo_{rec}(\ell) \triangleq$$

$$\log(\ell, x);$$

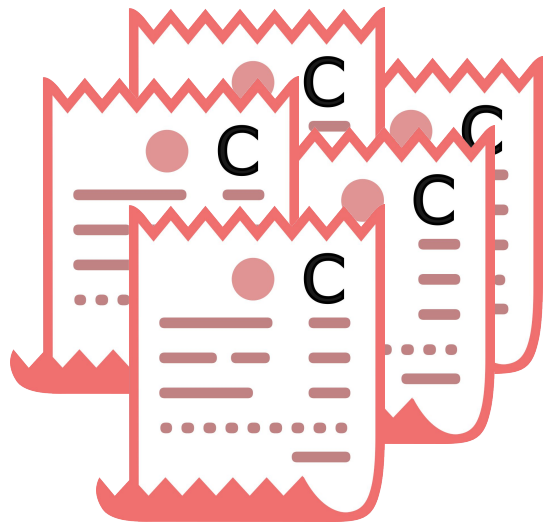
$$\{ \dots \}$$

$$\{\ell \mapsto buf\}$$

$$\log_{rec}(\ell, v) \triangleq$$

$$\vdots$$


$$\{\ell \mapsto v :: buf\}$$



$$\{\ell \mapsto buf\}$$

```

echorec( $\ell$ )  $\triangleq$ 
  let  $x :=$  getc() in
    log( $\ell$ ,  $x$ );
    putc(x)
  { ... }

```

$$\{\ell \mapsto buf\}$$

```

logrec( $\ell$ ,  $v$ )  $\triangleq$ 
  ⋮

```



$$\{\ell \mapsto v :: buf\}$$

$\{ True \}$

$putc_{rec}(v);$

$\{ True \}$

$\{ True \}$

$getc_{rec}();$

$\{ r, \exists v. r = v \}$

$\{ \ell \mapsto buf \}$

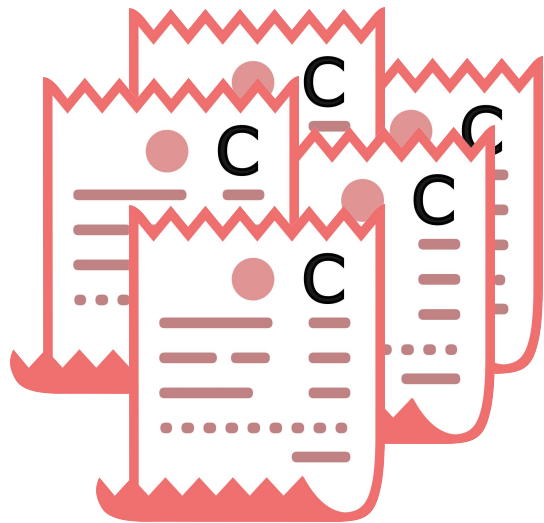
$echo_{rec}(\ell) \triangleq$

let $x := getc()$ in

log(ℓ, x);

putc(x)

$\{ \dots \}$



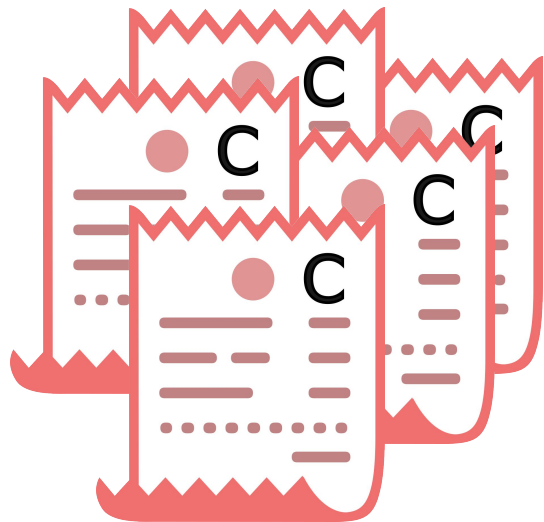
$\{ \ell \mapsto buf \}$

$log_{rec}(\ell, v) \triangleq$

$\vdots$



$\{ \ell \mapsto v :: buf \}$

$\{ True \}$ $putc_{rec}(v);$ $\{ True \}$ $\{ True \}$ $getc_{rec}();$ $\{ r, \exists v. r = v \}$ $\{ \ell \mapsto buf \}$ $echo_{rec}(\ell) \triangleq$ $let\ x := getc() in$ $log(\ell, x);$ $putc(x)$  $\{ \exists v. \ell \mapsto v :: buf \}$  $\{ \ell \mapsto buf \}$ $log_{rec}(\ell, v) \triangleq$ $\vdots$  $\{ \ell \mapsto v :: buf \}$

$\{ True \}$

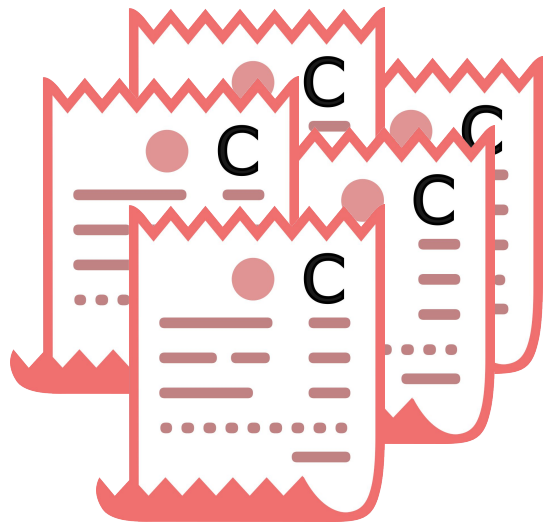
$putc_{rec}(v);$

$\{ True \}$

$\{ True \}$

$getc_{rec}();$

$\{ r, \exists v. r = v \}$



$\{ \ell \mapsto buf \}$

$log_{rec}(\ell, v) \triangleq$
:
:



$\{ \ell \mapsto v :: buf \}$

$\{ \ell \mapsto buf \}$

$echo_{rec}()$

$let x = getc_{rec}();$

$log(\ell, x);$

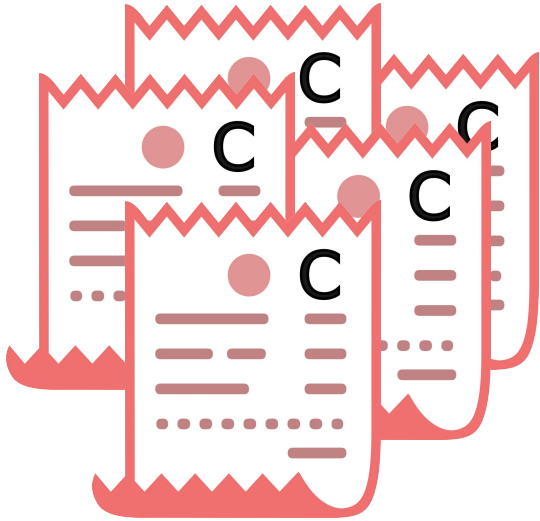
$putc(x)$



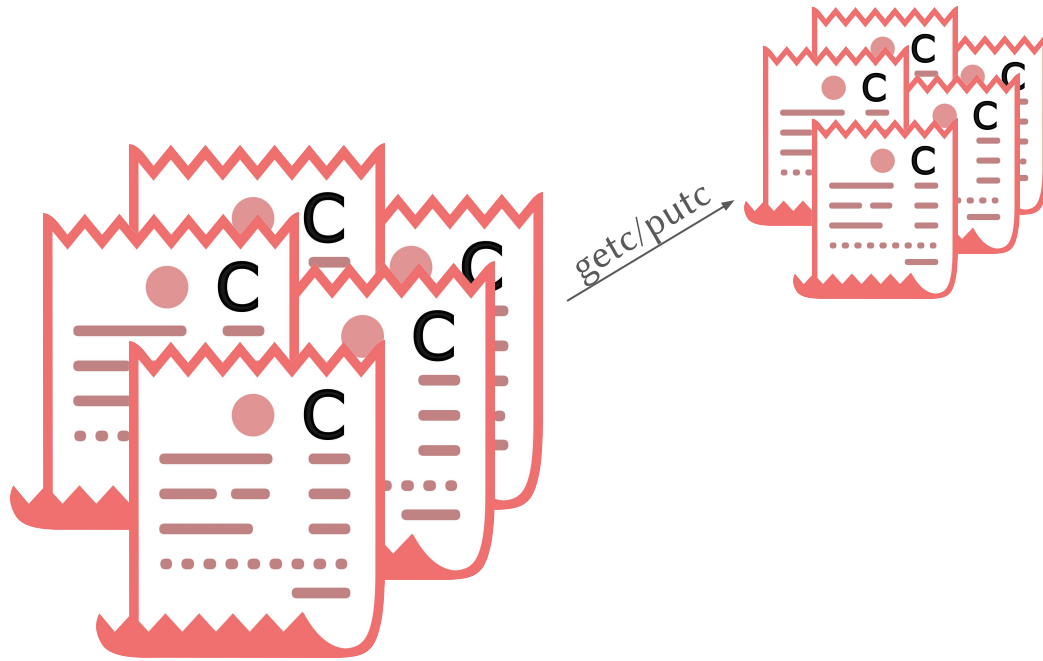
$\{ \exists v. \ell \mapsto v :: buf \}$

Wahahahaha!

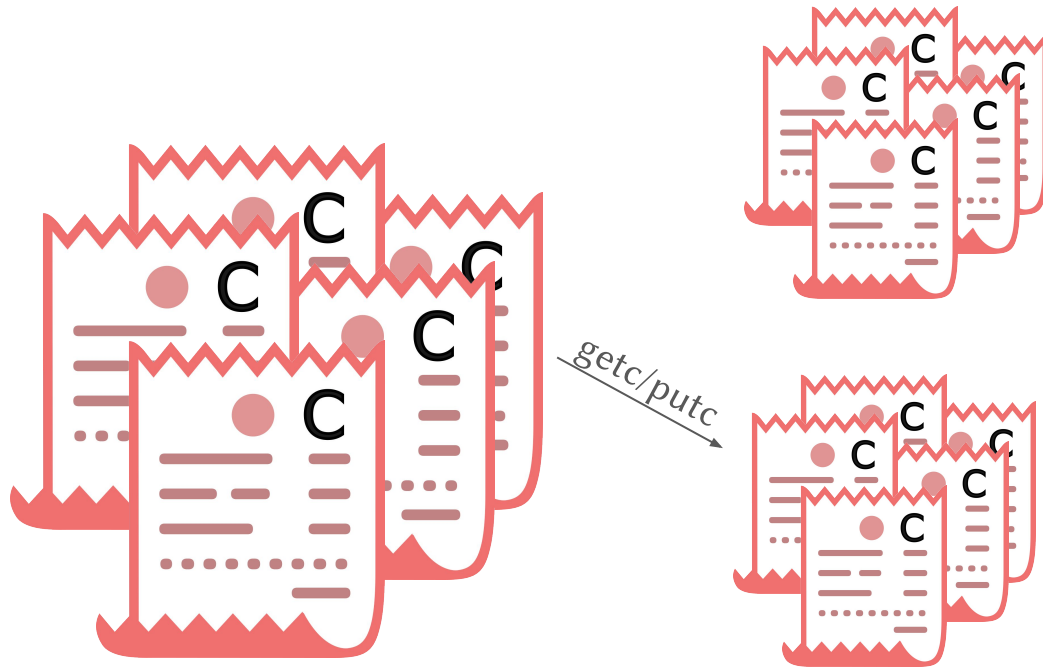
Real Programs call external code



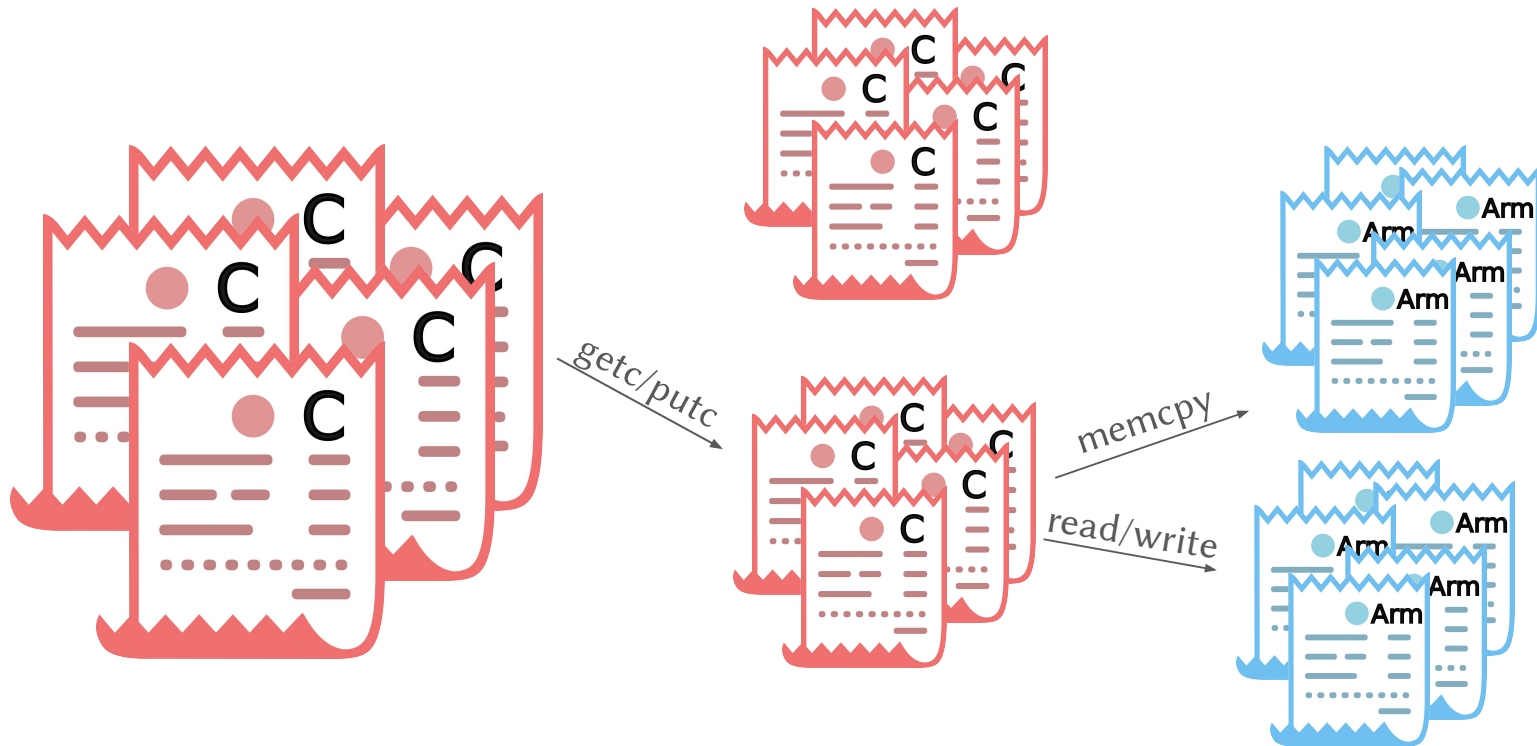
Real Programs call external code



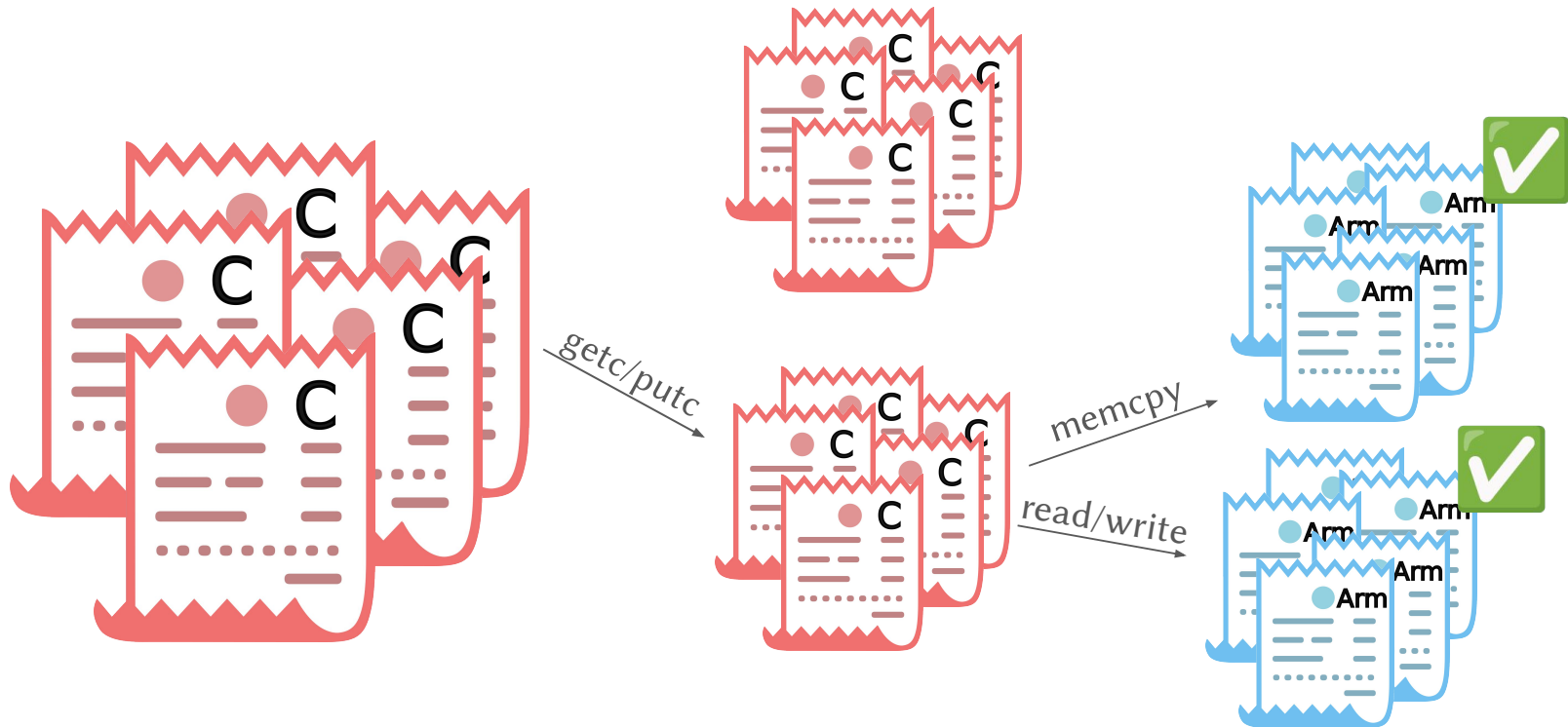
Real Programs call external code



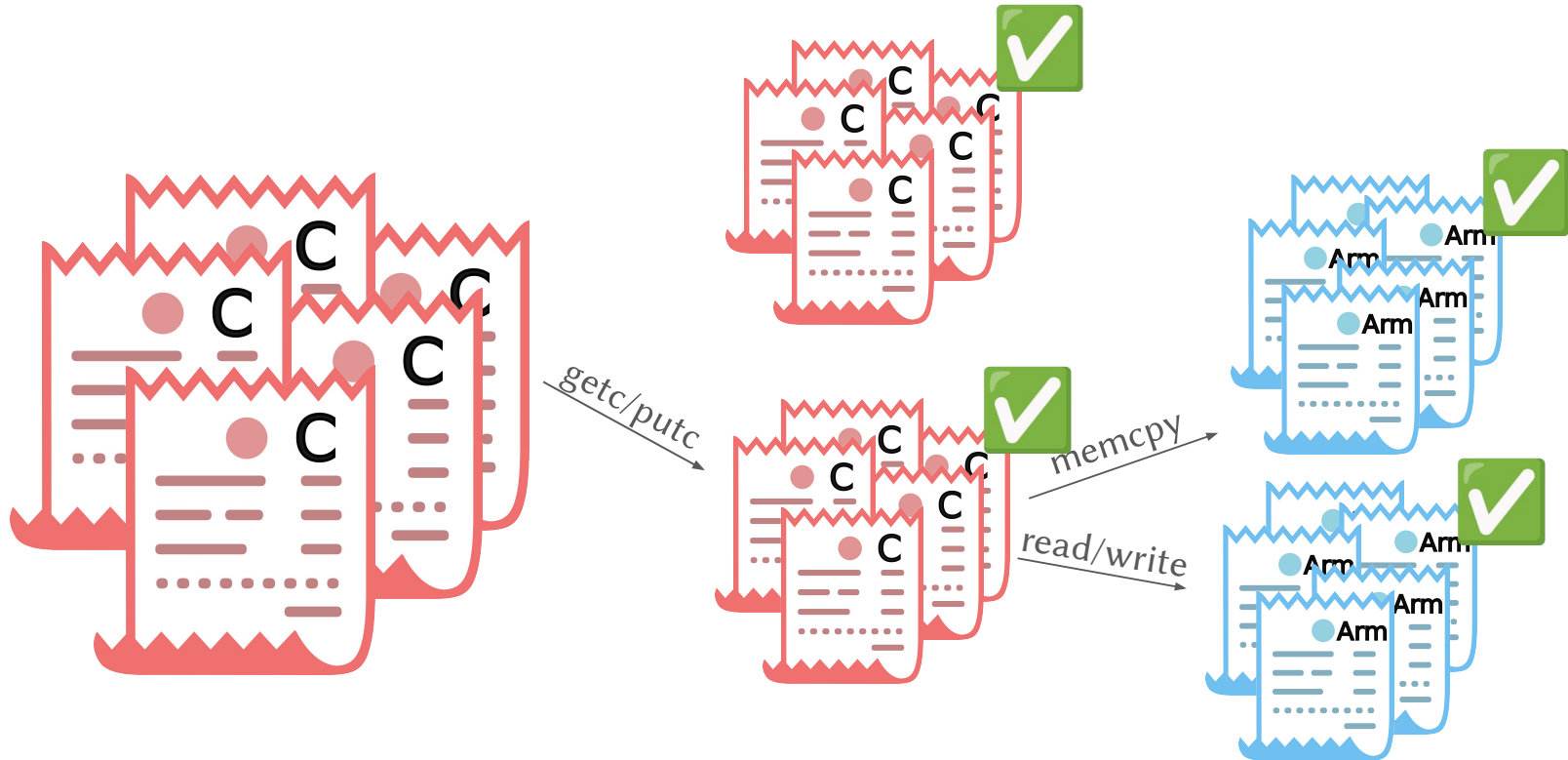
Real Programs call external code in different languages



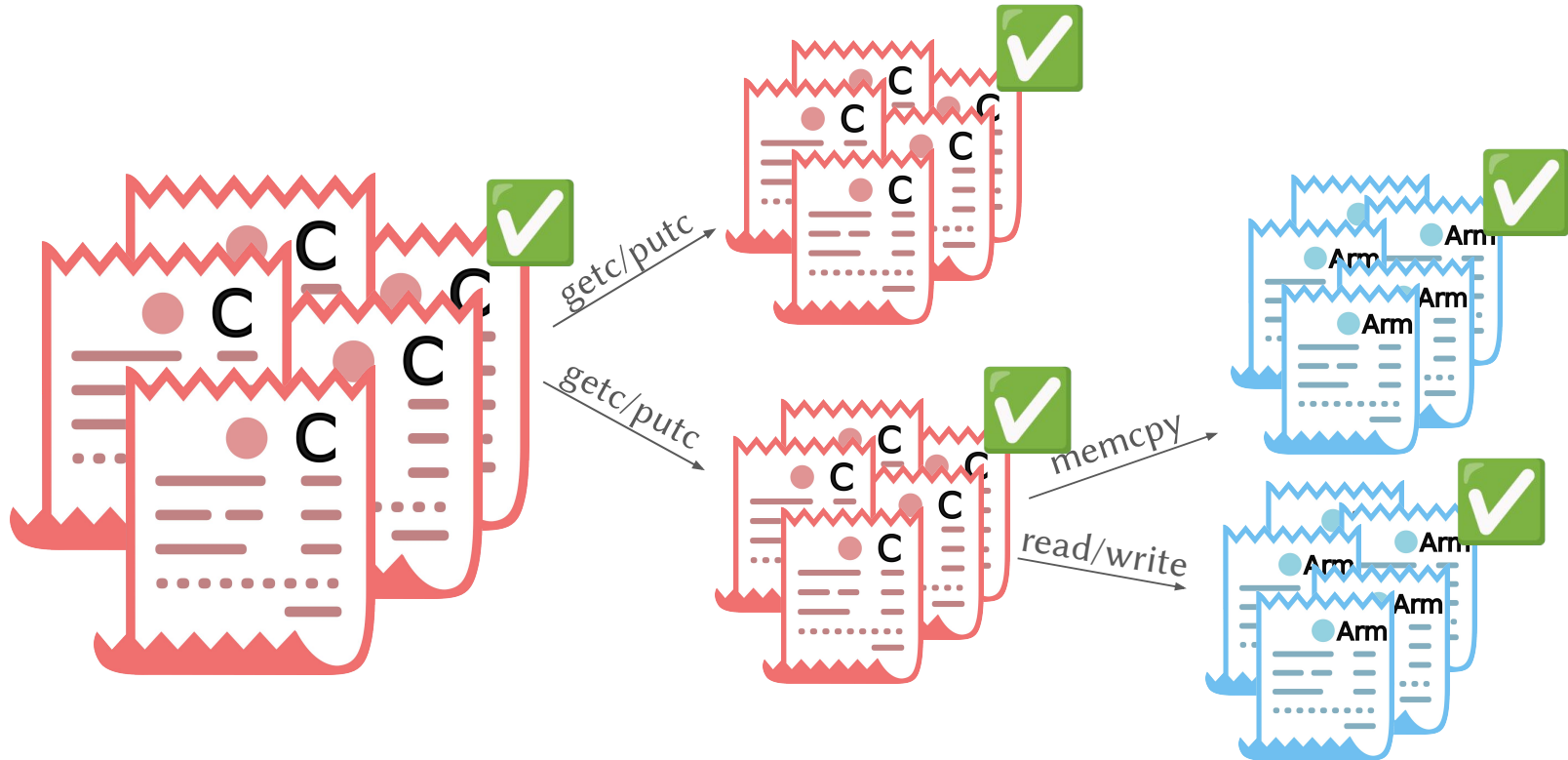
Real Programs call external code in different languages



Real Programs call external code in different languages



Real Programs call external code in different languages



Real Programs call external code in different languages



Key Challenge

How to handle external calls

Key Challenges: Multi-language Program Logic



Semantics of multi-language programs



Specifying external calls



Reasoning about multi-language programs



Proving refinements



Reconciling heterogeneous languages

Key Challenges: Multi-language Program Logic



Semantics of multi-language programs



Specifying external calls



Reasoning about multi-language programs



Proving refinements



Reconciling heterogeneous languages

Key Challenges: Multi-language Program Logic



Semantics of multi-language programs



Specifying external calls



Reasoning about multi-language programs



Proving refinements



Reconciling heterogeneous languages

Key Challenges: Multi-language Program Logic



Semantics of multi-language programs



Specifying external calls



Reasoning about multi-language programs



Proving refinements



Reconciling heterogeneous languages

Key Challenges: Multi-language Program Logic



Semantics of multi-language programs



Specifying external calls



Reasoning about multi-language programs



Proving refinements



Reconciling heterogeneous languages

DimSum: A Decentralized Approach to Multi-language Semantics and Verification

DimSum: A Decentralized Approach to Multi-language Semantics and Verification

MICHAEL SAMMLER, MPI-SWS, Germany

SIMON SPIES, MPI-SWS, Germany

YOUNGJU SONG, MPI-SWS, Germany

EMANUELE D'OSUALDO, MPI-SWS, Germany

ROBBERT KREBBERS, Radboud University Nijmegen, The Netherlands

DEEPAK GARG, MPI-SWS, Germany

DEREK DREYER, MPI-SWS, Germany

Prior work on multi-language program verification has achieved impressive results, including the compositional verification of complex compilers. But the existing approaches to this problem impose a variety of restrictions on the overall structure of multi-language programs (e.g., fixing the source language, fixing the set of involved languages, fixing the memory model, or fixing the semantics of interoperation). In this paper, we explore the problem of how to avoid such global restrictions.

Concretely, we present **DimSum**: a new, *decentralized* approach to multi-language semantics and verification, which we have implemented in the Coq proof assistant. Decentralization means that we can define and reason about languages independently from each other (as independent *modules* communicating via events), but also combine and translate between them when necessary (via a library of combinators).

We apply DimSum to a high-level imperative language **Rec** (with an abstract memory model and function calls), a low-level assembly language **Asm** (with a concrete memory model, arbitrary jumps, and syscalls), and a mathematical specification language **Spec**. We evaluate DimSum on two case studies: an **Asm** library extending **Rec** with support for pointer comparison, and a coroutine library for **Rec** written in **Asm**. In both cases, we show how DimSum allows the **Asm** libraries to be abstracted to **Rec**-level specifications, despite the behavior of the **Asm** libraries not being syntactically expressible in **Rec** itself. We also verify an optimizing multi-pass compiler from **Rec** to **Asm**, showing that it is compatible with these **Asm** libraries.

DimSum: A Decentralized Approach to Multi-language Semantics and Verification

- *Decentralized* multi-language semantics as communicating modules
- Refinement as key notion of verification
- Three languages:
 - **Rec**: High-level, impure language: function call and block-based memory
 - **Asm**: Assembly language: jumps, systems calls, and flat memory
 - **Pseudo**: Pseudo-code language for abstract mathematical algorithms
- Multi-language programs through combinators at semantic level
 - $\llbracket \text{echo} \cup \text{getc} \rrbracket \oplus \llbracket \text{putc} \rrbracket \leq \llbracket \text{echo_rw_pseudo} \rrbracket$
 - $\lceil \llbracket \text{echo} \cup \text{getc} \rrbracket \rceil_{\text{ra}} \oplus \llbracket \text{putc} \rrbracket \leq \llbracket \text{echo_sys_pseudo} \rrbracket$

DimSum: A Decentralized Approach to Multi-language Semantics and Verification

- *Decentralized* multi-language semantics as communicating modules
- Refinement as key notion of verification
- Three languages:
 - **Rec**: High-level, impure language: function call and block-based memory
 - **Asm**: Assembly language: jumps, systems calls, and flat memory
 - **Pseudo**: Pseudo-code language for abstract mathematical algorithms
- Multi-language programs through combinators at semantic level
 - $\llbracket \text{echo} \cup \text{getc} \rrbracket \oplus \llbracket \text{putc} \rrbracket \leq \llbracket \text{echo_rw_pseudo} \rrbracket$
 - $\lceil \llbracket \text{echo} \cup \text{getc} \rrbracket \rceil_{\text{ra}} \oplus \llbracket \text{putc} \rrbracket \leq \llbracket \text{echo_sys_pseudo} \rrbracket$

DimSum: A Decentralized Approach to Multi-language Semantics and Verification

- *Decentralized* multi-language semantics as communicating modules
- Refinement as key notion of verification
- Three languages:
 - **Rec**: High-level, impure language: function call and block-based memory
 - **Asm**: Assembly language: jumps, systems calls, and flat memory
 - **Pseudo**: Pseudo-code language for abstract mathematical algorithms
- Multi-language programs through combinators at semantic level
 - $\llbracket \text{echo } u \text{ getc} \rrbracket \oplus \llbracket \text{putc} \rrbracket \leq \llbracket \text{echo_rw_pseudo} \rrbracket$
 - $\lceil \llbracket \text{echo } u \text{ getc} \rrbracket \rceil_{\text{ra}} \oplus \llbracket \text{putc} \rrbracket \leq \llbracket \text{echo_sys_pseudo} \rrbracket$

DimSum: A Decentralized Approach to Multi-language Semantics and Verification

- *Decentralized* multi-language semantics as communicating modules
- Refinement as key notion of verification
- Three languages:
 - **Rec**: High-level, impure language: function call and block-based memory
 - **Asm**: Assembly language: jumps, systems calls, and flat memory
 - **Pseudo**: Pseudo-code language for abstract mathematical algorithms
- Multi-language programs through combinators at semantic level
 - $\llbracket \text{echo} \cup \text{getc} \rrbracket \oplus \llbracket \text{putc} \rrbracket \leq \llbracket \text{echo_rw_pseudo} \rrbracket$
 - $\llbracket \llbracket \text{echo} \cup \text{getc} \rrbracket \rrbracket_{\text{ra}} \oplus \llbracket \text{putc} \rrbracket \leq \llbracket \text{echo_sys_pseudo} \rrbracket$

DimSum: A Decentralized Approach to Multi-language Semantics and Verification

- *Decentralized* multi-language semantics as communicating modules
- Refinement as key notion of verification
- Three languages:
 - **Rec**: High-level, impure language: function call and block-based memory
 - **Asm**: Assembly language: jumps, systems calls, and flat memory
 - **Pseudo**: Pseudo-code language for abstract mathematical algorithms
- Multi-language programs through combinators at semantic level
 - $\llbracket \text{echo} \cup \text{getc} \rrbracket \oplus \llbracket \text{putc} \rrbracket \leq \llbracket \text{echo_rw_pseudo} \rrbracket$
 - $\lceil \llbracket \text{echo} \cup \text{getc} \rrbracket \rceil_{\text{ra}} \oplus \llbracket \text{putc} \rrbracket \leq \llbracket \text{echo_sys_pseudo} \rrbracket$



Key Challenges



Semantics of multi-language programs



Specifying external calls



Reasoning about multi-language programs



Proving refinements



Reconciling heterogeneous languages

Specifying External Calls

`echorec()` $\triangleq$ `let` $x := \text{getc}()$ `in` `putc(x)`

Specifying External Calls

$\frac{?}{\underline{P_{\text{echo}}} \vdash \text{echo}_{\text{rec}}() \triangleq \text{let } x := \text{getc}() \text{ in } \text{putc}(x)}$

Specifying External Calls



Key Idea

Abstract Call as Specification Construct

(x)

Specifying External Calls

$\text{echo}_{\text{rec}}() \triangleq \text{let } x := \text{getc}() \text{ in putc}(x)$

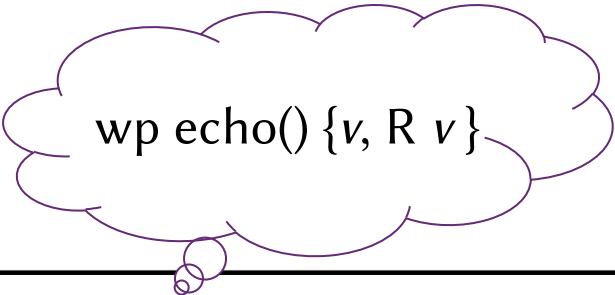
SPEC-ECHO

$v \leftarrow \text{echo}()_r; R(v)$

Specifying External Calls

$\text{echo}_{\text{rec}}() \triangleq \text{let } x := \text{getc}() \text{ in } \text{putc}(x)$

SPEC-ECHO



$\text{wp echo}() \{v, R\ v\}$

$v \leftarrow \text{echo}()_r; R(v)$

Specifying External Calls

$\text{echo}_{\text{rec}}() \triangleq \text{let } x := \text{getc}() \text{ in } \text{putc}(x)$

SPEC-ECHO

$$\frac{\text{echo} \hookrightarrow_r \text{echo}_{\text{rec}} \quad v \leftarrow \text{getc}()_r; v' \leftarrow \text{putc}(v)_r; R(v')}{v \leftarrow \text{echo}()_r; R(v)}$$

Specifying External Calls

$\text{echo}_{\text{rec}}() \triangleq \text{let } x := \text{getc}() \text{ in putc}(x)$

SPEC-ECHO

$$\frac{\text{echo} \hookrightarrow_r \text{echo}_{\text{rec}} \quad \mathbf{v} \leftarrow \text{getc}()_r; \mathbf{v}' \leftarrow \text{putc}(\mathbf{v})_r; R(\mathbf{v}')}{\mathbf{v} \leftarrow \text{echo}()_r; R(\mathbf{v})}$$

Specifying External Calls

$\text{echo}_{\text{rec}}() \triangleq \text{let } x := \text{getc}() \text{ in } \text{putc}(x)$

SPEC-ECHO

$$\frac{\text{echo} \hookrightarrow_r \text{echo}_{\text{rec}} \quad \mathbf{v} \leftarrow \text{getc}()_r; \mathbf{v}' \leftarrow \text{putc}(\mathbf{v})_r; R(\mathbf{v}')}{\mathbf{v} \leftarrow \text{echo}()_r; R(\mathbf{v})}$$

Specifying External Calls

$\text{echo}_{\text{rec}}() \triangleq \text{let } x := \text{getc}() \text{ in putc}(x)$

SPEC-ECHO

$$\frac{\text{echo} \hookrightarrow_r \text{echo}_{\text{rec}} \quad \mathbf{v} \leftarrow \text{getc}()_r; \mathbf{v}' \leftarrow \text{putc}(\mathbf{v})_r; R(\mathbf{v}')}{\mathbf{v} \leftarrow \text{echo}()_r; R(\mathbf{v})}$$

Specifying External Calls

SPEC-ECHO

$$\frac{\text{echo} \hookrightarrow_r \text{echo}_{\text{rec}} \quad \mathbf{v} \leftarrow \text{getc}()_r; \mathbf{v}' \leftarrow \text{putc}(\mathbf{v})_r; R(\mathbf{v}')}{\mathbf{v} \leftarrow \text{echo}()_r; R(\mathbf{v})}$$

$$\text{putc}_{\text{rec}}(\mathbf{v}) \triangleq \text{local } x[1]; x \leftarrow \mathbf{v}; \text{write}(x, 1)$$

Specifying External Calls

SPEC-ECHO

$$\frac{\text{echo} \hookrightarrow_r \text{echo}_{\text{rec}} \quad \mathbf{v} \leftarrow \text{getc}()_r; \mathbf{v}' \leftarrow \text{putc}(\mathbf{v})_r; R(\mathbf{v}')}{\mathbf{v} \leftarrow \text{echo}()_r; R(\mathbf{v})}$$

$$\text{putc}_{\text{rec}}(\mathbf{v}) \triangleq \text{local } x[1]; x \leftarrow \mathbf{v}; \text{write}(x, 1)$$

SPEC-PUTC

$$\frac{\text{putc} \hookrightarrow_r \text{putc}_{\text{rec}} \quad \forall \ell. \ell \mapsto \mathbf{v} \multimap (\mathbf{v}' \leftarrow \text{write}(\ell, 1)_r; \ell \mapsto \mathbf{v} * R(\mathbf{v}'))}{\mathbf{v}' \leftarrow \text{putc}(\mathbf{v})_r; R(\mathbf{v}')}$$

Specifying External Calls

SPEC-ECHO

$$\frac{\text{echo} \hookrightarrow_r \text{echo}_{\text{rec}} \quad \mathbf{v} \leftarrow \text{getc}()_r; \mathbf{v}' \leftarrow \text{putc}(\mathbf{v})_r; R(\mathbf{v}')}{\mathbf{v} \leftarrow \text{echo}()_r; R(\mathbf{v})}$$

$$\text{putc}_{\text{rec}}(\mathbf{v}) \triangleq \text{local } x[1]; x \leftarrow \mathbf{v}; \text{write}(x, 1)$$

SPEC-PUTC

$$\frac{\text{putc} \hookrightarrow_r \text{putc}_{\text{rec}} \quad \forall \ell. \ell \mapsto \mathbf{v} \multimap (\mathbf{v}' \leftarrow \text{write}(\ell, 1)_r; \ell \mapsto \mathbf{v} \multimap R(\mathbf{v}'))}{\mathbf{v}' \leftarrow \text{putc}(\mathbf{v})_r; R(\mathbf{v}')}$$

Specifying External Calls

SPEC-ECHO

$$\frac{\text{echo} \hookrightarrow_r \text{echo}_{\text{rec}} \quad \mathbf{v} \leftarrow \text{getc}()_r; \mathbf{v}' \leftarrow \text{putc}(\mathbf{v})_r; R(\mathbf{v}')}{\mathbf{v} \leftarrow \text{echo}()_r; R(\mathbf{v})}$$

$$\text{putc}_{\text{rec}}(\mathbf{v}) \triangleq \text{local } x[1]; x \leftarrow \mathbf{v}; \text{write}(x, 1)$$

SPEC-PUTC

$$\frac{\text{putc} \hookrightarrow_r \text{putc}_{\text{rec}} \quad \forall \ell. \ell \mapsto \mathbf{v} \multimap (\mathbf{v}' \leftarrow \text{write}(\ell, 1)_r; \ell \mapsto \mathbf{v} \multimap R(\mathbf{v}'))}{\mathbf{v}' \leftarrow \text{putc}(\mathbf{v})_r; R(\mathbf{v}')}$$

Specifying External Calls

SPEC-ECHO

$$\frac{\text{echo} \hookrightarrow_r \text{echo}_{\text{rec}} \quad \mathbf{v} \leftarrow \text{getc}(); \mathbf{v}' \leftarrow \text{putc}(\mathbf{v}); R(\mathbf{v}')}{\mathbf{v} \leftarrow \text{echo}(); R(\mathbf{v})}$$

SPEC-PUTC

$$\frac{\text{putc} \hookrightarrow_r \text{putc}_{\text{rec}} \quad \forall \ell. \ell \mapsto \mathbf{v} * (\mathbf{v}' \leftarrow \text{write}(\ell, 1); \ell \mapsto \mathbf{v} * R(\mathbf{v}'))}{\mathbf{v}' \leftarrow \text{putc}(\mathbf{v}); R(\mathbf{v})}$$

$$\text{getc}_{\text{rec}}() \triangleq \text{local } x[1]; \text{read}(x, 1); !x$$

Specifying External Calls

SPEC-ECHO

$$\frac{\text{echo} \hookrightarrow_r \text{echo}_{\text{rec}} \quad \mathbf{v} \leftarrow \text{getc}()_r; \mathbf{v}' \leftarrow \text{putc}(\mathbf{v})_r; R(\mathbf{v}')}{\mathbf{v} \leftarrow \text{echo}()_r; R(\mathbf{v})}$$

SPEC-PUTC

$$\frac{\text{putc} \hookrightarrow_r \text{putc}_{\text{rec}} \quad \forall \ell. \ell \mapsto \mathbf{v} * (\mathbf{v}' \leftarrow \text{write}(\ell, 1)_r; \ell \mapsto \mathbf{v} * R(\mathbf{v}'))}{\mathbf{v}' \leftarrow \text{putc}(\mathbf{v})_r; R(\mathbf{v}')}$$

$$\text{getc}_{\text{rec}}() \triangleq \text{local } x[1]; \text{read}(x, 1); !x$$

SPEC-GETC

$$\frac{\text{getc} \hookrightarrow_r \text{getc}_{\text{rec}} \quad \forall \ell \mathbf{v}. \ell \mapsto \mathbf{v} * (\text{read}(\ell, 1)_r; \exists \mathbf{v}'. \ell \mapsto \mathbf{v}' * R(\mathbf{v}'))}{\mathbf{v} \leftarrow \text{getc}()_r; R(\mathbf{v})}$$

Key Challenges



Semantics of multi-language programs



Specifying external calls



Reasoning about multi-language programs



Proving refinements



Reconciling heterogeneous languages

`echorec()`

`getcrec()`

`putcrec(v)`

`putcpseudo(v)`

`putcasm`

`readasm`

Rec

echo_{rec}()

Rec

`echorec()`

`getcrec()`

`putcrec(v)`



Rec

`echorec()`

`getcrec()`

`putcrec(v)`

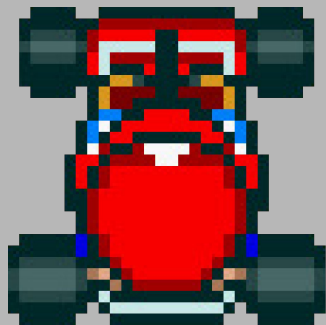


Rec

$\text{echo}_{\text{rec}}()$

$\text{getc}_{\text{rec}}()$

$\text{putc}_{\text{rec}}(v)$



SPEC-ECHO

$\text{echo} \hookrightarrow_r \text{echo}_{\text{rec}} \quad v \leftarrow \text{getc}()_r; v' \leftarrow \text{putc}(v)_r; R(v')$

$v \leftarrow \text{echo}()_r; R(v)$

Rec

`echorec()`

`getcrec()`

`putcrec(v)`



Rec



Pseudo

`echorec()`

`getcrec()`



Rec



Pseudo

`echorec()`

`getcrec()`



```
putcpseudo(v)  $\triangleq$   
  buf := buf ++ [v];  
  if * then  
    ℓ := alloc(|buf|);  
    ℓ ← buf;  
    write(ℓ, |buf|);  
    buf := []  
  return 0
```

Rec



Pseudo

echo_{rec}()

getc_{rec}()

putc_{pseudo}(v)



Rec



Pseudo

echo_{rec}()

getc_{rec}()



putc_{pseudo}(v)

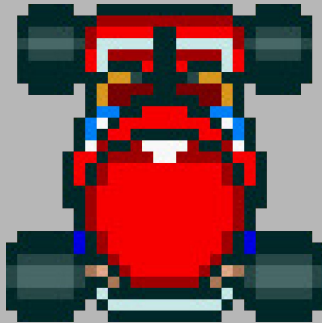
Rec



!putc(**v**)

echo_{rec}()

getc_{rec}()



Pseudo

putc_{pseudo}(**v**)

Rec



Pseudo

!putc(v)

echo_{rec}()

putc_{pseudo}(v)

getc_{rec}()



Rec



Pseudo

`echorec()`

`?putc(v)`

`putcpseudo(v)`

`getcrec()`



Rec



Pseudo

`echorec()`

`getcrec()`

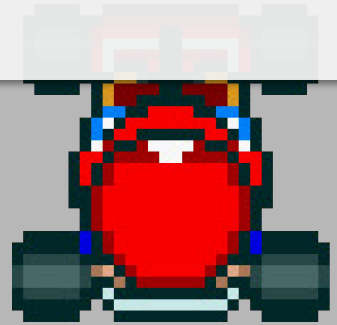
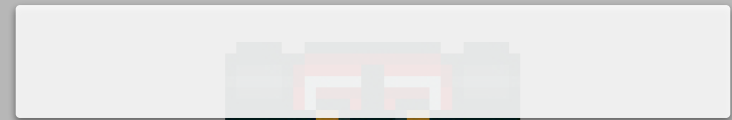
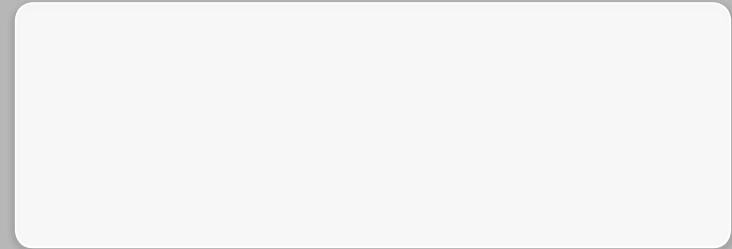
`putcpseudo(v)`



echo_{rec}() Rec



getc_{rec}()



Pseudo putc_{pseudo}(v)

SPEC-ECHO

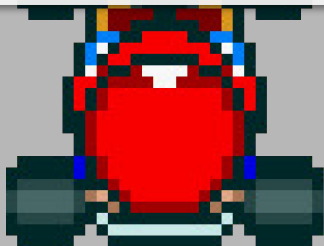
$\text{echo} \hookrightarrow_r \text{echo}_{\text{rec}} \quad v \leftarrow \text{getc}()_r; v' \leftarrow \text{putc}(v)_r; R(v')$

$v \leftarrow \text{echo}()_r; R(v)$

$\uparrow \Pi_{\oplus_r}$

$\Pi[_ \oplus_r \bullet] \Leftarrow \text{clock}; Q$

$v' \leftarrow \text{putc}(v)_r; R(v')$



SWITCH-CALL

$f \not\rightarrow_r$ $!^\Pi \text{Call}(f, \bar{v}); \text{⌚}; v' \leftarrow ?^\Pi \text{Return}; R(v')$

$\text{getc}_{\text{rec}}()$

$v \leftarrow f(\bar{v})_r; R$

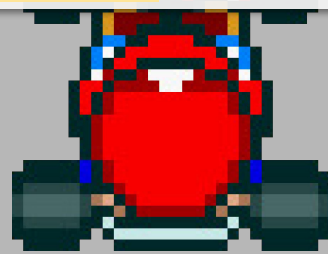
$\text{tc}_{\text{pseudo}}(v)$

$\uparrow^\Pi_{\oplus_r}$

$\Pi[_\oplus_r \bullet] \Leftarrow \text{⌚}; Q$

$\vdash$

$v' \leftarrow \text{putc}(v)_r; R(v')$



SWITCH-CALL

$f \not\rightarrow_r$ $!^\Pi \text{Call}(f, \bar{v}); \text{⌚}; v' \leftarrow ?^\Pi \text{Return}; R(v')$

$\text{getc}_{\text{rec}}()$

$v \leftarrow f(\bar{v})_r; R$

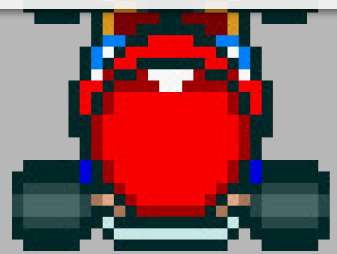
$\text{tc}_{\text{pseudo}}(v)$

$\uparrow^\Pi_{\oplus_r}$

$\Pi[_\oplus_r \bullet] \Leftarrow \text{⌚}; Q$

$\vdash$

$!^\Pi \text{Call}(\text{putc}, v); \text{⌚}; v' \leftarrow ?^\Pi \text{Return}; R(v')$



echo_{rec}()

getc_{rec}()

SWITCH-REDIRECT

$$\frac{\Pi' \Leftarrow P' \quad \Pi \Leftarrow P \multimap !^{\Pi} e \Longrightarrow P'}{!^{\Pi} e; P}$$

putc_{pseudo}(v)

$\uparrow^{\Pi}$
 $\oplus_r$

$\Pi[_\oplus_r \bullet] \Leftarrow \text{clock}; Q$

⊢

$!^{\Pi} \text{Call}(\text{putc}, v); \text{clock}; v' \leftarrow ?^{\Pi} \text{Return}; R(v')$



echo_{rec}()

getc_{rec}()

SWITCH-REDIRECT

$$\frac{\Pi' \Leftarrow P' \quad \Pi \Leftarrow P \multimap !^{\Pi} e \xRightarrow{\Pi'} P'}{!^{\Pi} e; P}$$

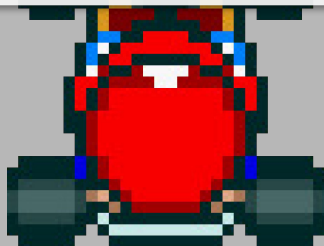
putc_{pseudo}(v)

$\uparrow_{\oplus_r}^{\Pi}$

$$\Pi[\bullet \oplus_r _] \Leftarrow \text{Return}; R(v')$$

⊢

$$!^{\Pi[\bullet \oplus_r _]} \text{Call}(\text{putc}, v) \xRightarrow{\Pi[_ \oplus_r \bullet]} \text{Return}; Q$$



echo_{rec}()

getc_{rec}()

SWITCH-LINK-L-TO-R

$f \in d_2$

$\uparrow_{\oplus_r}^{\Pi}$

$\downarrow_{\oplus_r}^{\Pi}$

$(\text{Call}(f, \bar{v})) \rightarrow^* P$

do

putc_{pseudo}(v)

$$\frac{}{!^{\Pi[\bullet \oplus_r _]} \text{Call}(f, \bar{v}) \xRightarrow{\Pi[_ \oplus_r \bullet]} P}$$

$\uparrow_{\oplus_r}^{\Pi}$

$\Pi[\bullet \oplus_r _] \Leftarrow \text{⌚}; v' \leftarrow ?^{\Pi} \text{Return}; R(v')$

⊢

$$!^{\Pi[\bullet \oplus_r _]} \text{Call}(\text{putc}, v) \xRightarrow{\Pi[_ \oplus_r \bullet]} \text{⌚}; Q$$



echo_{rec}()

getc_{rec}()

SWITCH-LINK-L-TO-R

$f \in d_2 \quad \uparrow_{\oplus_r}^{\Pi} \quad \downarrow_{\oplus_r}^{\Pi} (\text{Call}(f, \bar{v})) \rightarrow^* P$

$\frac{}{!^{\Pi[\bullet \oplus_r _]} \text{Call}(f, \bar{v}) \xRightarrow{\Pi[_ \oplus_r \bullet]} P}$

do

putc_{pseudo}(v)

$\downarrow_{\oplus_r}^{\Pi} (\text{Call}(\text{putc}, v))$

$\Pi[\bullet \oplus_r _] \Leftarrow \text{!}; v' \leftarrow ?^{\Pi} \text{Return}; R(v')$

⊢

!; Q



echo_r

getc_{rec}()

PSEUDO-WAIT-CALL

$$\frac{f, \bar{v} \leftarrow ?^{\Pi} \text{Call}; v \leftarrow f(\bar{v})_p; !^{\Pi} \text{Return}(v); \bullet; P}{\bullet; P}$$

putc_{pseudo}(v)

$$\downarrow_{\oplus_r}^{\Pi} (\text{Call}(\text{putc}, v))$$
$$\Pi[\bullet \oplus_r _] \Leftarrow \bullet; v' \leftarrow ?^{\Pi} \text{Return}; R(v')$$

⊢

•; Q



echo_r

getc_{rec}()

PSEUDO-WAIT-CALL

$$\frac{f, \bar{v} \leftarrow ?^{\Pi} \text{Call}; v \leftarrow f(\bar{v})_p; !^{\Pi} \text{Return}(v); \text{⌚}; P}{\text{⌚}; P}$$

putc_{pseudo}(v)

$\downarrow_{\oplus_r}^{\Pi} (\text{Call}(\text{putc}, v))$

$\Pi[\bullet \oplus_r _] \Leftarrow \text{⌚}; v' \leftarrow ?^{\Pi} \text{Return}; R(v')$

⊢

$f, \bar{v} \leftarrow ?^{\Pi[_ \oplus_r \bullet]} \text{Call}; v \leftarrow f(\bar{v})_p; \dots$



echo_{rec}()

getc_{rec}()

SWITCH-LINK-L-IN

$\downarrow_{\oplus_r}^{\Pi} (?Call(f, \bar{v}))$

$\uparrow_{\oplus_r}^{\Pi} \rightarrow * P f \bar{v}$

$f, \bar{v} \leftarrow ?^{\Pi}[_{\oplus_r} \bullet] Call; P$

do

putc_{pseudo}(v)

$\downarrow_{\oplus_r}^{\Pi} (Call(putc, v))$

$\Pi[\bullet \oplus_r _] \Leftarrow \text{clock}; v' \leftarrow ?^{\Pi} Return; R(v')$

⊢

$f, \bar{v} \leftarrow ?^{\Pi}[_{\oplus_r} \bullet] Call; v \leftarrow f(\bar{v})_p; \dots$



getc_{rec}()

echo_{rec}()

SWITCH-LINK-L-IN
$$\frac{\downarrow_{\oplus_r}^{\Pi} (? \text{Call}(f, \bar{v})) \quad \uparrow_{\oplus_r}^{\Pi} \rightarrow * P f \bar{v}}{f, \bar{v} \leftarrow ?^{\Pi}[_{\oplus_r} \bullet] \text{Call}; P}$$

do putc_{pseudo}(v)

$$\uparrow_{\oplus_r}^{\Pi}$$

$$\Pi[\bullet \oplus_r _] \Leftarrow \text{Clock}; v' \leftarrow ?^{\Pi} \text{Return}; R(v')$$

⊢

$$v' \leftarrow \text{putc}(v)_p; \dots$$



Key Challenges



Semantics of multi-language programs



Specifying external calls



Reasoning about multi-language programs



Proving refinements



Reconciling heterogeneous languages

Rec_{TGT}



Pseudo_{TGT}



Pseudo

Write only
previously
read chars

echo_{rec}()

putc_{pseudo}(v)

echo_rw_{pseudo}()

getc_{rec}()



Rec_{TGT}



Pseudo_{TGT}



Pseudo_{SRC}

echo_{rec}()

putc_{pseudo}(v)

echo_rw_{pseudo}()

getc_{rec}()



Rec_{TGT}



Pseudo



Pseudo_{SRC}

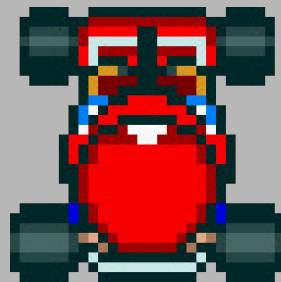
!write(*ℓ*, 1)

echo_{rec}()

putc_{pseudo}(*v*)

echo_rw_{pseudo}()

getc_{rec}()



Rec_{TGT}



echo_{rec}()

getc_{rec}()

Pseudo_{TGT}

putc_{pseudo}(v)



Pseudo_{SRC}

!write(*l*, 1)

echo_rw_{pseudo}()



Key Challenges



Semantics of multi-language programs



Specifying external calls



Reasoning about multi-language programs



Proving refinements



Reconciling heterogeneous languages

Rec_{TGT}

$\lceil - \rceil$ $r \Leftrightarrow a$



Asm_{TGT}



Pseudo_{SRC}

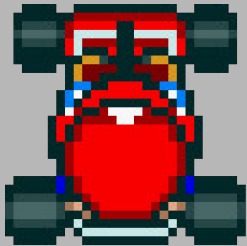
echo_{rec}()

getc_{rec}()

putc_{asm}

read_{asm}

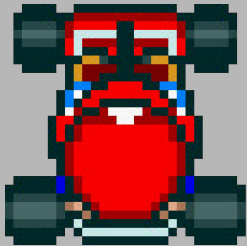
echo_sys_{pseudo}()



Rec_{TGT}

echo_{rec}()

getc_{rec}()



[-]

$r \rightleftharpoons a$

$\oplus$

Asm_{TGT}

putc_{asm}

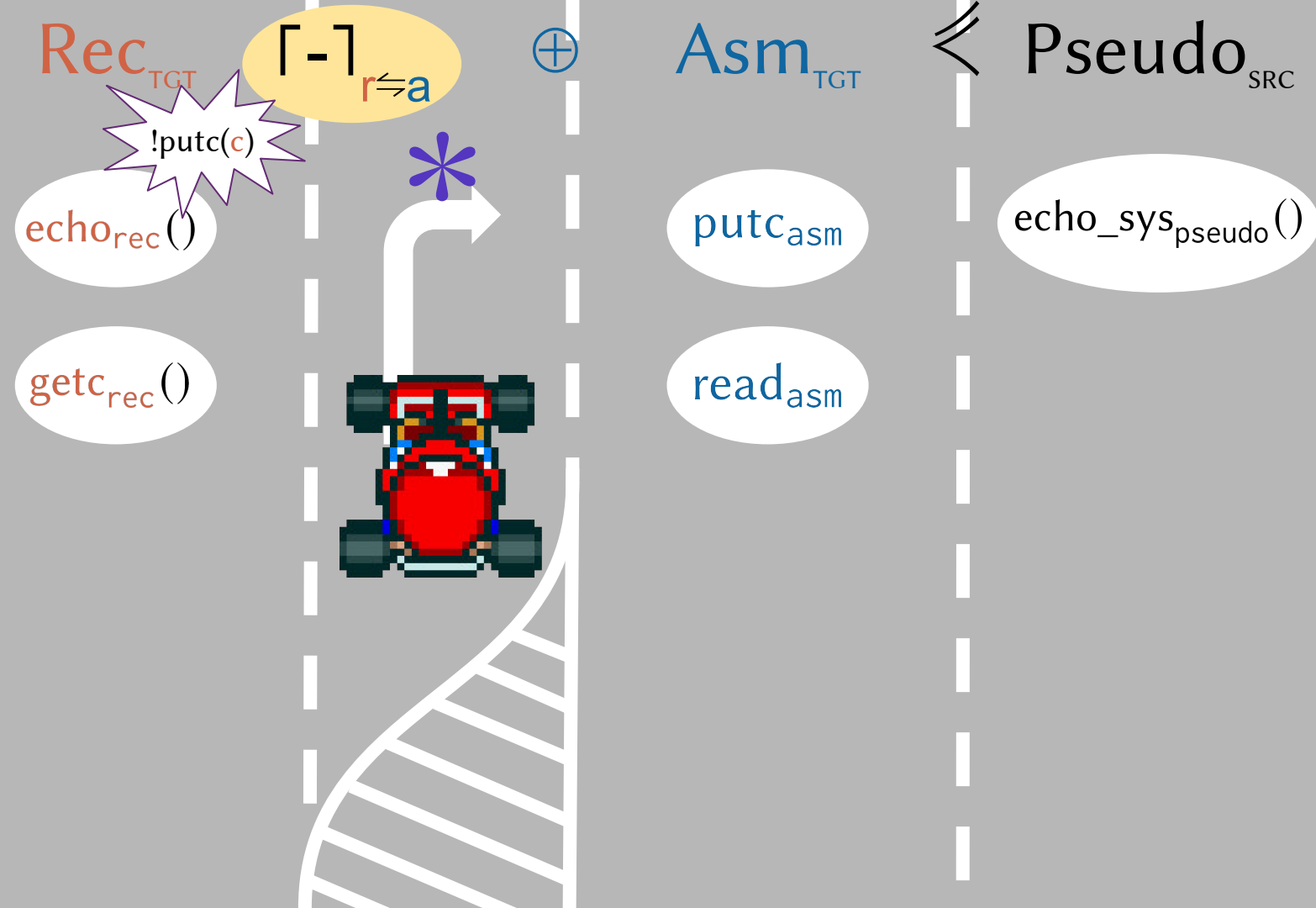
read_{asm}

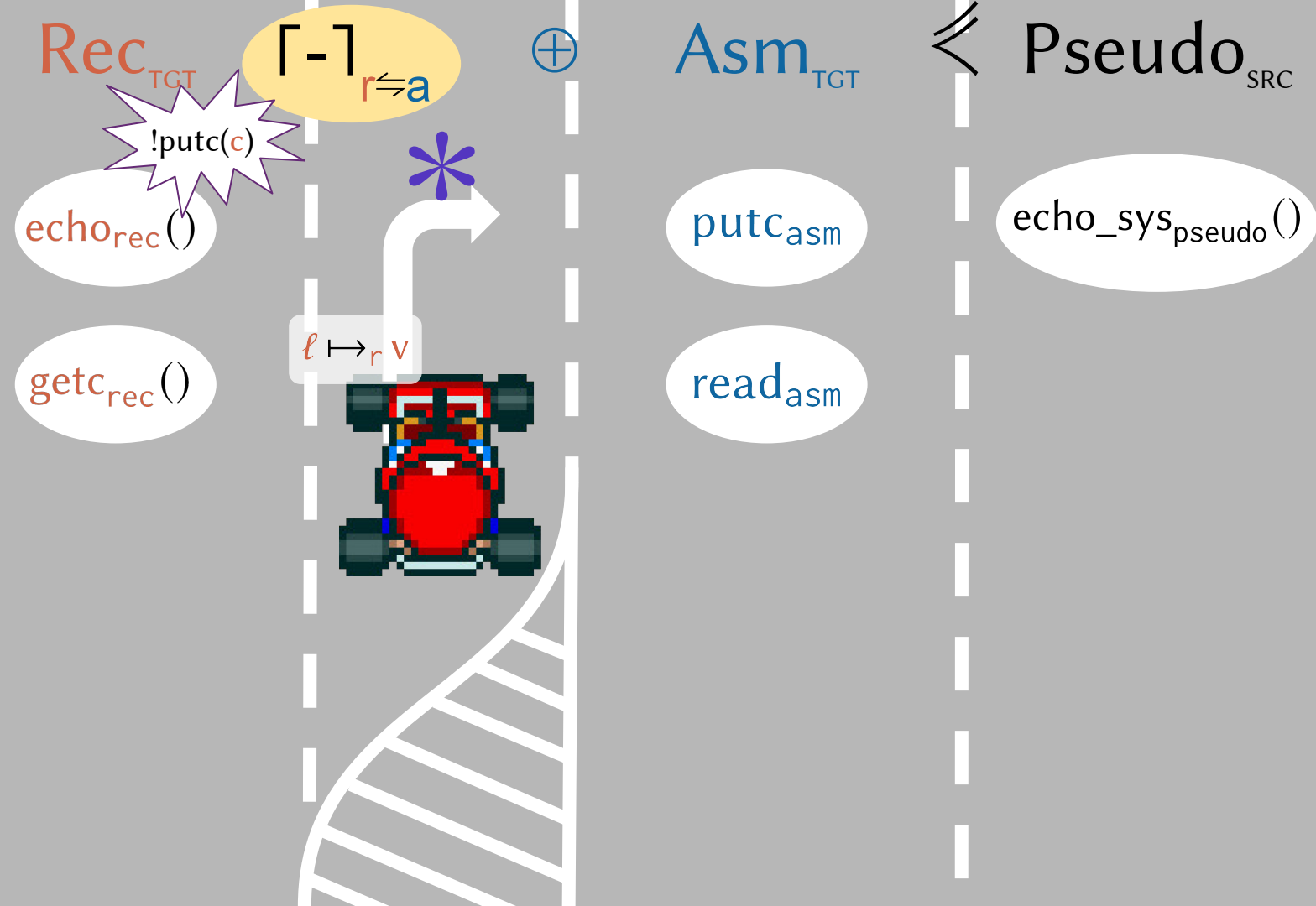
$\ll$

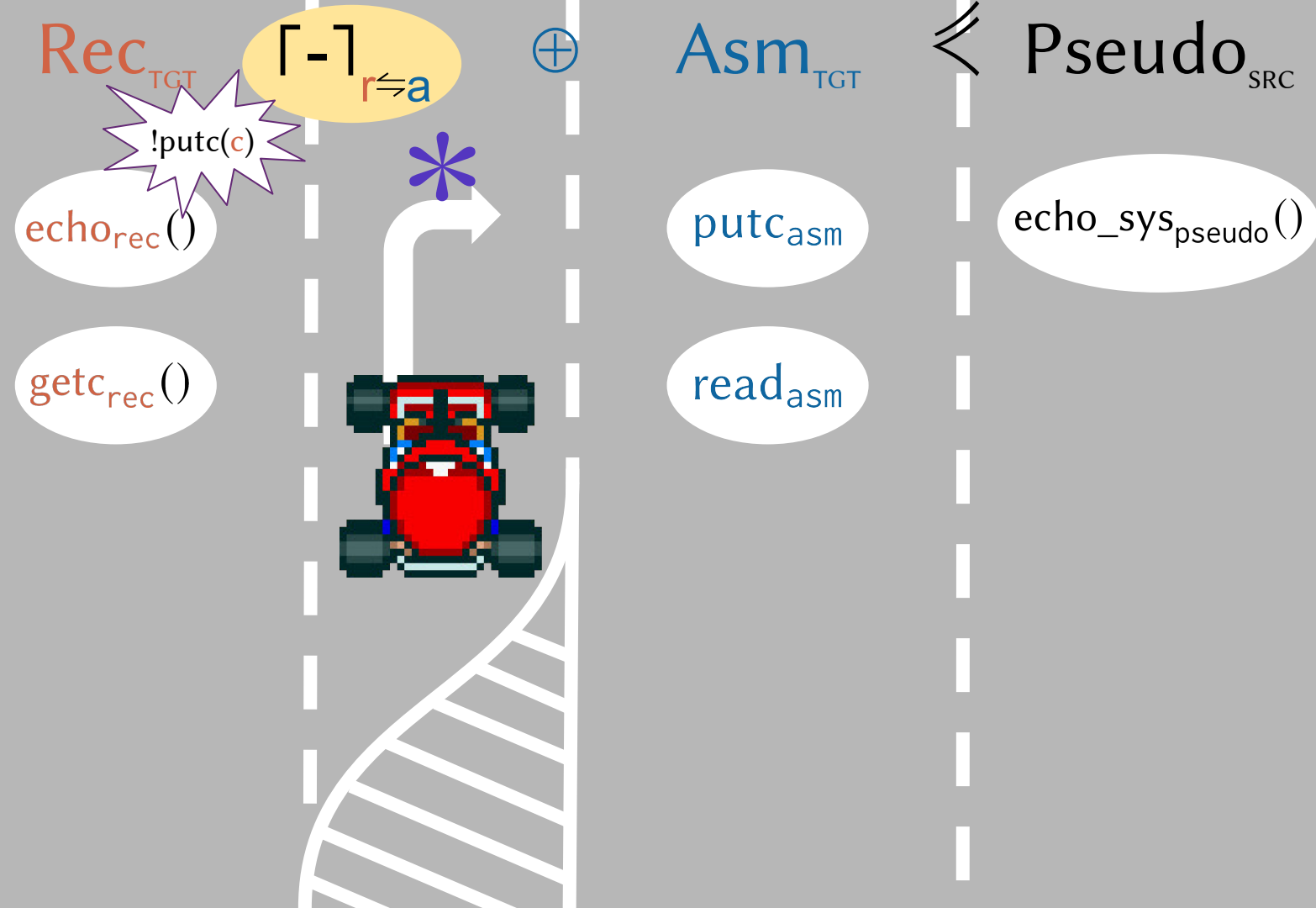
Pseudo_{SRC}

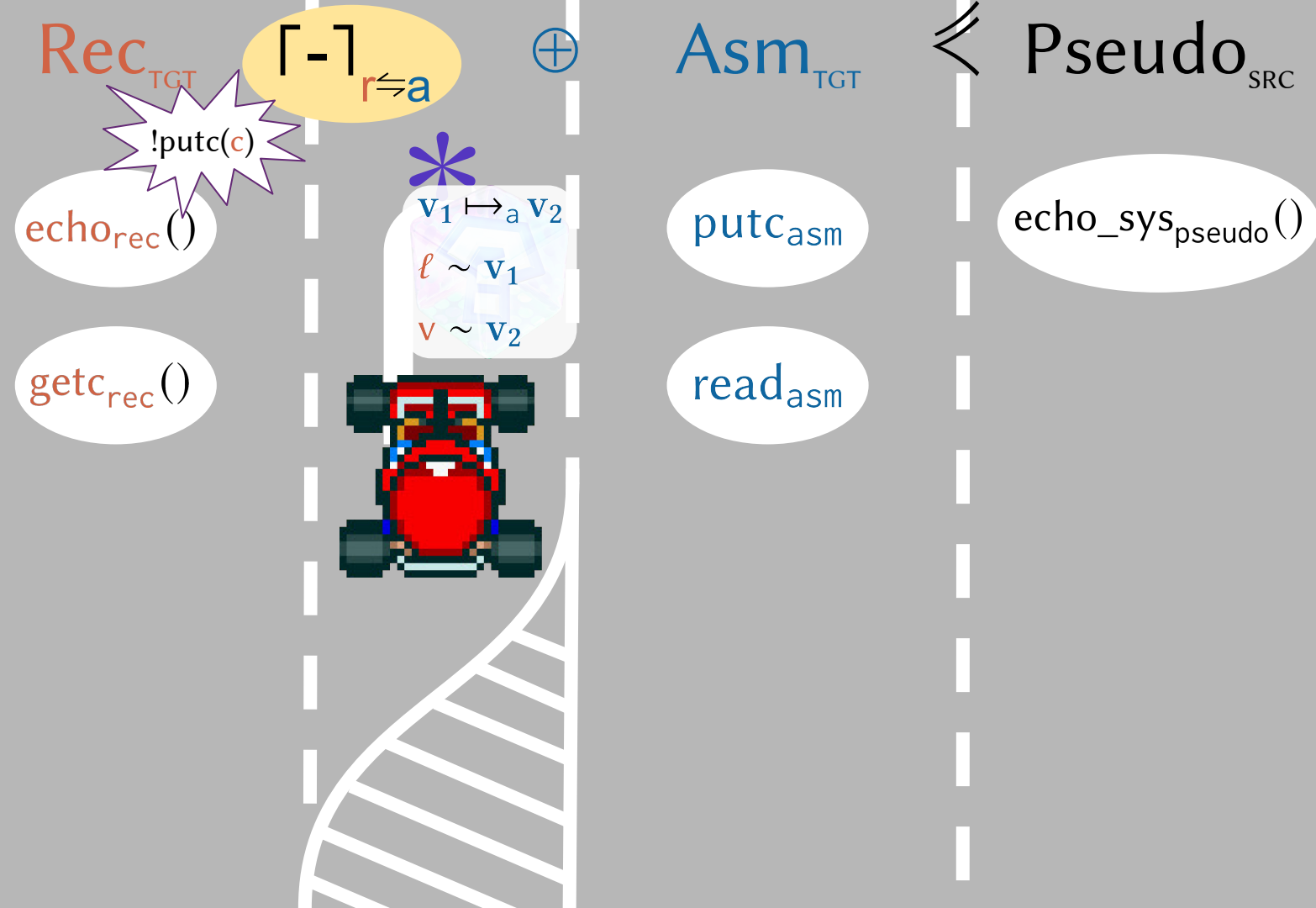
echo_sys_{pseudo}()











Rec_{TGT}

$\lceil - \rceil$

$r \Leftarrow a$



Asm_{TGT}



Pseudo_{SRC}

echo_{rec}()

getc_{rec}()



!jump(putc_{asm})

putc_{asm}

read_{asm}

echo_sys_{pseudo}()

`echorec()`



`getcrec()`

`putcrec(v)`

`putcpseudo(v)`

`putcasm`

`readasm`

Key Ideas



Semantics of multi-language programs



Specifying external calls



Reasoning about multi-language programs



Proving refinements



Reconciling heterogeneous languages

Key Ideas



DimSum's multi-language semantics



Specifying external calls



Reasoning about multi-language programs



Proving refinements



Reconciling heterogeneous languages

Key Ideas



DimSum's multi-language semantics



Abstract Calls as specification construct



Reasoning about multi-language programs



Proving refinements



Reconciling heterogeneous languages

Key Ideas



DimSum's multi-language semantics



Abstract Calls as specification construct



Lanes to reason about multi-language programs



Proving refinements



Reconciling heterogeneous languages

Key Ideas



DimSum's multi-language semantics



Abstract Calls as specification construct



Lanes to reason about multi-language programs



Lanes to prove refinements



Reconciling heterogeneous languages

Key Ideas



DimSum's multi-language semantics



Abstract Calls as specification construct



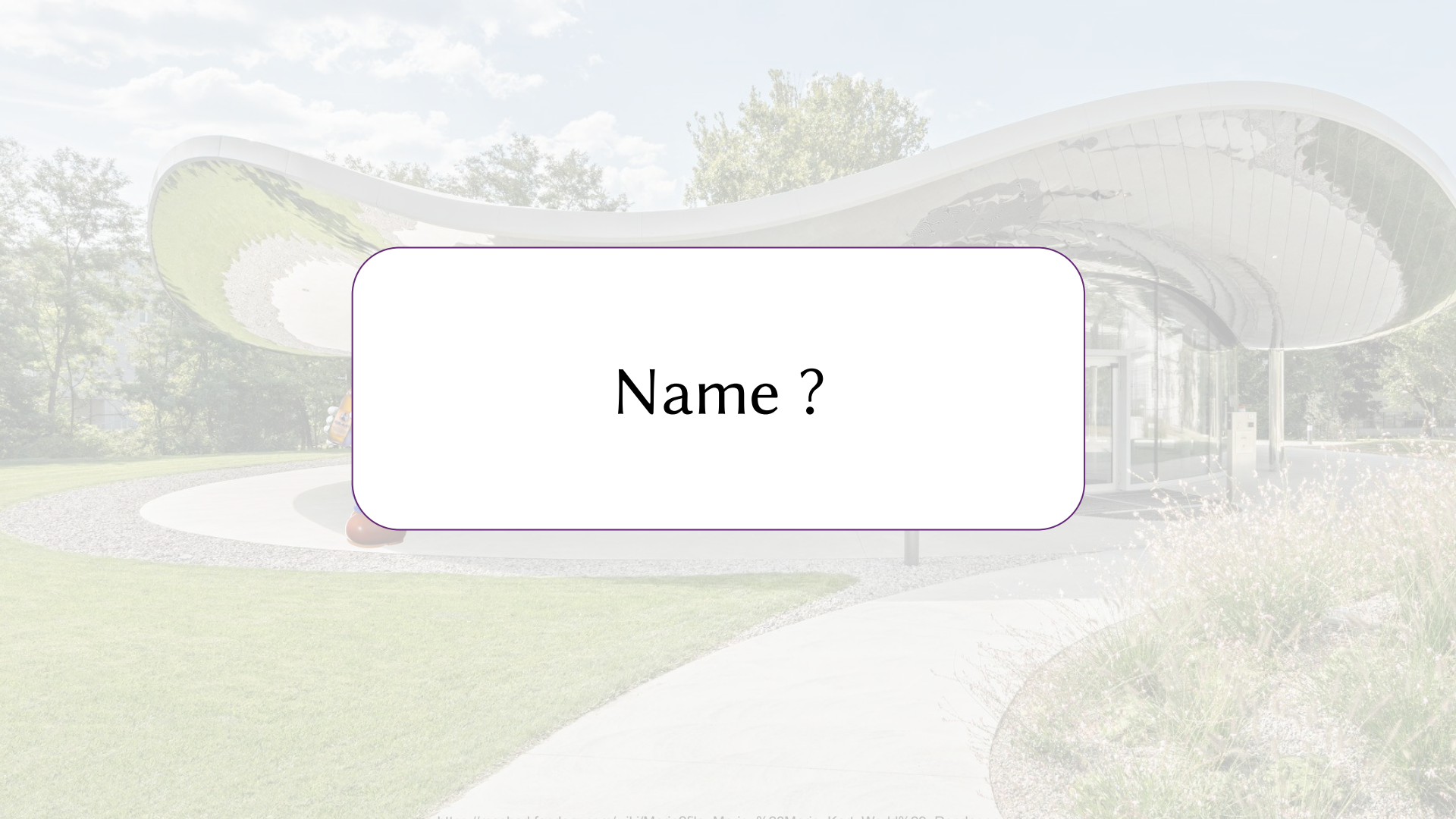
Lanes to reason about multi-language programs



Lanes to prove refinements



Lanes to reconcile heterogeneous languages



Name ?